

SYSTEM REQUIREMENT SPECIFICATION (SRS)

2.1 Introduction

2.1.1 Purpose

The purpose of this document is to provide a comprehensive description of the **API Monitor SaaS** system. It outlines the functional and non-functional requirements, design constraints, and system interfaces. This document serves as a guide for developers, administrators, and stakeholders (including project guides and evaluators) to understand the system's architecture and capabilities.

2.1.2 Scope

The **API Monitor SaaS** is a specialized platform designed to provide real-time health monitoring for web APIs. It allows users to register, manage their API endpoints, visualize performance metrics (uptime, response times), and receive instant notifications via email when an endpoint fails. The system includes a multi-layer security engine that uses DNS resolution and VirusTotal integration to reject dummy or fake API endpoints, ensuring only legitimate services are monitored. The system aims to minimize downtime for developers and businesses by providing an automated monitoring engine.

2.1.3 Acronyms and Abbreviations

Acronym	Definition
SRS	Software Requirement Specification
API	Application Programming Interface
SaaS	Software as a Service
JWT	JSON Web Token
SMTP	Simple Mail Transfer Protocol
RLS	Row Level Security
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
UI	User Interface
OTP	One-Time Password
DNS	Domain Name System
VT	VirusTotal

2.1.4 Overview

The document is organized into several sections:

- **Section 2.1** introduces the project and its scope.
- **Section 2.2** describes the overall system, product functions, and operating environment.
- **Section 2.3** details the functional requirements and external interfaces.
- **Non-Functional Requirements** covers performance, security, and quality attributes.
- **Section 2.4** summarizes the system's advantages.
- **Section 2.5** outlines future scope.

2.2 Overall Description

2.2.1 Product Functions

The main functions of the system include:

- **User Authentication:** Secure registration and login using JWT.
- **Endpoint Management:** CRUD operations for API endpoints to be monitored.
- **Automated Monitoring:** Background engine performing health checks every 60 seconds.
- **Alerting System:** Automated SMTP email notifications on status changes (down/up).
- **Data Visualization:** Interactive charts for response time and uptime analysis.
- **Profile Management:** User profile updates and security settings.
- **Dummy API Detection:** A 3-layer validation engine (Static Blacklist, DNS Resolution, VirusTotal Reputation) that rejects fake, mock, or non-existent API endpoints during the addition phase.
- **API Key Validation:** Pre-flight validation of user-supplied API keys before storing them, with pattern matching for known test key formats.

2.2.2 Typical Data Flow

1. **User Input:** User adds an API URL and configuration via the frontend.
2. **Endpoint Validation:** The backend runs a 3-layer check on the URL:
 - Layer 1: Static blacklist of known dummy hosts (e.g., mockapi.io, dummyjson.com).
 - Layer 2: DNS resolution check – if the domain does not resolve, it is immediately rejected.
 - Layer 3: VirusTotal API domain reputation check – unknown, malicious, or suspicious domains are rejected.
3. **Database Storage:** If the URL passes all checks, the Flask backend stores the endpoint details in Supabase (Postgres).
4. **Monitoring Cycle:** The APScheduler-based engine fetches active endpoints and pings them.
5. **Logging:** Results (status code, latency) are logged into the monitoring_logs table.
6. **Notification:** If failure criteria are met, the Alert Service triggers an SMTP email to the user.
7. **Visualization:** The dashboard fetches logs via the REST API to render Chart.js graphs.

2.2.3 User Classes and Characteristics

- **Standard Users:** Developers or system administrators who want to monitor their APIs. They have access to their own dashboard and endpoints.
- **Administrators:** Users with elevated permissions to manage the global system settings and user base (Future Scope).

2.2.4 Operating Environment

- **Server Side:** Python 3.10+, Flask, Supabase (Cloud Database).
- **Client Side:** Any modern web browser (Chrome, Firefox, Safari, Edge).
- **Hosting:** Render, Vercel, or Heroku.

2.2.5 Design and Implementation Constraints / Assumptions

- The system depends on the availability of the Supabase API.
- Monitoring frequency is fixed at 60 seconds unless configured otherwise.
- SMTP delivery depends on the provider (e.g., Gmail) and correct App Password configuration.
- The VirusTotal API has a rate limit of 4 requests per minute on the free tier.

2.2.6 How Dynamic Website Works

The system uses a **Full-Stack Decoupled Architecture**:

- The **Frontend** (HTML/CSS/JS) is a Single Page Application (SPA) style setup that communicates with the backend via fetch requests.
- The **Backend** (Flask) acts as a REST API server, handling authentication, database logic, and URL reputation checks.
- The **Background Engine** runs asynchronously to ensure monitoring continues even without active user sessions.
- The **URL Reputation Service** integrates with VirusTotal API v3 and local DNS checks to validate endpoint legitimacy before adding them to the database.

2.2.7 Assumptions and Dependencies

- **Assumed Browser:** HTML5 and ES6 compatibility.
- **Internet Discovery:** Users must have a stable internet connection to access the dashboard.
- **Email Service:** Assumes the user provides a valid email address for alerts.
- **VirusTotal API:** Requires a valid API key for domain reputation analysis.

2.3 Functional Requirements

2.3.1 Core Functionality Description

The system provides a seamless experience for monitoring APIs. Users can define endpoints (URL, Method, Interval), and the system ensures they are reachable. The dashboard provides a "NOC-style" view of all monitored services. Before any endpoint is added, a multi-layer validation engine checks the URL against a static blacklist, performs DNS resolution, and queries VirusTotal for domain reputation to prevent dummy or mock APIs from being monitored.

2.3.2 External Interface Requirements

2.3.2.1 User Interface

- **Landing Page:** Modern centered glassmorphic login/registration card.
- **Dashboard:** Sidebar navigation with dynamic content area, health status cards, and responsive charts.
- **Modals:** For adding/editing endpoints with animated error messages (shake animation) for invalid or dummy APIs.
- **Error Alerts:** In-modal animated alerts with slide-in and shake effects for immediate user feedback on validation failures.

2.3.2.2 Hardware Interfaces

- No specialized hardware required. Standard Server/PC with NIC.

2.3.2.3 Software Interfaces

- **Web Browser:** Chrome 90+, Firefox 88+.
- **Database:** Supabase (PostgreSQL).
- **Backend Framework:** Flask (Python).
- **Libraries:** Chart.js for visualization, APScheduler for tasks.
- **External APIs:** VirusTotal API v3 for domain reputation analysis.

2.3.2.4 Communication Interfaces

- **HTTP/HTTPS:** For client-server communication and API pings.
- **SMTP:** For sending critical failure alerts.
- **VirusTotal REST API:** For querying domain reputation data during endpoint validation.

2.3.3 Detailed Functional Requirements

2.3.3.1 Web Pages Identified

- **landing.html:** Entry point for guest users.
- **index.html:** Login and Registration portal.
- **dashboard.html:** The main control center showing global stats.
- **api-endpoints.html:** Detailed list of monitored APIs with management controls and "Add Endpoint" modal.
- **profile.html:** User account and notification settings.

2.3.3.2 Dummy API Detection Module

The system implements a 3-layer security gate for URL validation:

- **Layer 1 – Static Blacklist:** A curated list of known mock API providers (mockapi.io, dummyjson.com, beeceptor.com, etc.) is maintained on the server. Any URL matching these hosts is instantly rejected.
- **Layer 2 – DNS Resolution:** The system attempts to resolve the domain via DNS. If the domain does not exist (e.g., data.nexoraapi.io), it is rejected immediately with the message "Domain does not exist (DNS resolution failed)."
- **Layer 3 – VirusTotal Reputation:** For domains that pass DNS, the system queries the VirusTotal API v3 (/api/v3/domains/{host}) to check
 - Reputation score (negative = blocked).
 - Malicious/suspicious detection count.
 - Analysis coverage (zero coverage = blocked as unknown).

Non-Functional Requirements

- **Security:**
 - Password hashing using bcrypt.
 - Authorization via JWT in headers.
 - Row Level Security (RLS) on all database tables to prevent data leakage between users.
 - API key masking in all responses (only first 4 and last 4 characters shown).
 - Pre-flight API key pattern matching to reject known test key formats.
 - VirusTotal-powered domain reputation checks to prevent fake endpoints.
- **Performance:**
 - API health check responses logged in less than 500ms.
 - Dashboard page load less than 2 seconds.
 - Static blacklist and DNS checks complete in less than 100ms.
 - VirusTotal check completes within 10 seconds (with timeout fallback).
- **Scalability:**
 - Supabase handles concurrent database connections reliably.
 - Flask backend is stateless for horizontal scaling.
- **Availability:**
 - Background monitoring engine ensures 24/7 pings.
- **Usability:**
 - Clean, premium design with intuitive navigation.
 - Animated error messages (slide-in + shake) for failed validations.

2.4 Advantages of the System

- **Real-time Awareness:** Users know immediately when their API is down.
- **Data-Driven Insights:** Uptime and latency charts help identify performance regressions.
- **Low Barrier to Entry:** Easy to set up with existing endpoints.
- **Secure by Design:** Implementation of RLS ensures user data privacy.
- **Anti-Dummy Protection:** Industry-first 3-layer validation (Blacklist + DNS + VirusTotal) prevents users from monitoring fake or mock APIs.
- **Animated UX Feedback:** Premium shake and slide-in animations for validation errors provide clear, immediate user feedback.

2.5 Future Enhancements

- **Multi-region Pings:** Monitor APIs from different global locations.
- **Slack/Discord Webhooks:** Notification support for dev teams.
- **AI Diagnostics:** Automatically analyze root causes of failures (e.g., DNS, Timeout, 500 Error).
- **Mobile Application:** Native iOS/Android app for on-the-go alerts.
- **Custom Blacklist Management:** Allow users to add their own blacklisted domains via the Admin Panel.